

Task 11 – Terraform Preparation Documentation

Twenty CRM AWS Infrastructure (VPC, EC2, ECR)

1. Objective

Prepare Terraform configuration to provision the AWS infrastructure needed for the Twenty CRM application, limited to:

- **VPC** – reuse the account's existing/default VPC (no new VPC created)
- **EC2** – the application server
- **ECR** – the container image repository

This task is **preparation and validation only**. `terraform apply` (and, per the latest scope, `terraform plan`) are **not** run — only `terraform init` and `terraform validate`.

2. Project Structure

```
terraform-crm/
├─ main.tf                # Terraform + AWS provider
├─ configuration
│  ├─ variables.tf        # All configurable input variables
│  └─ vpc.tf              # Data sources for the existing
default VPC/subnets
├─ ec2.tf                 # Security group + EC2 instance for
the app
├─ ecr.tf                 # ECR repository + lifecycle policy
├─ outputs.tf             # Outputs (VPC id, instance IP, ECR
URL, etc.)
└─ terraform.tfvars.example # Example variable values (copy to
```

```
terraform.tfvars)
├── .gitignore                # Excludes state, .terraform/, real
tfvars
└── README.md                # Usage instructions
```

Terraform automatically loads every `.tf` file in a directory, so the config is split by concern (provider, variables, vpc, ec2, ecr, outputs) rather than kept in one large file — this is standard Terraform project layout and keeps each resource type easy to find and review.

3. File-by-File Explanation

`main.tf`

- Declares the required Terraform version (`>= 1.5.0`) and pins the AWS provider to `~> 5.0`.
- Configures the AWS provider to use `var.aws_region` (defaults to `us-east-1`).
- Applies `default_tags` so every resource is automatically tagged with `Project`, `Environment`, and `ManagedBy` without repeating tags in every resource block.
- Contains a **commented-out** S3 remote backend block, for future use if the state needs to be shared across a team instead of kept locally.

`variables.tf`

- Declares every configurable value used elsewhere in the project: `region`, `project/environment` names, `EC2 instance type`, `key pair name`, `SSH CIDR`, `app port`, `root volume size`, `ECR repository name`, `tag mutability`, `scan-on-push`, and `lifecycle expiry days`.
- Nothing is hardcoded into the resource files — all of it flows through variables with sensible defaults, so the same config can be reused for different environments by just changing `terraform.tfvars`.
- Includes a `validation` block on `ecr_image_tag_mutability` to reject any value other than `MUTABLE/IMMUTABLE` at plan time.

vpc.tf

- Uses **data sources**, not resources, for networking:
 - `data.aws_vpc.default` looks up the AWS account's existing default VPC.
 - `data.aws_subnets.default` looks up all subnets belonging to that VPC.
 - `data.aws_availability_zones.available` lists AZs available in the region, for future extension (e.g. multi-AZ deployment).
- This satisfies the "use existing/default AWS VPC setup" requirement — no `aws_vpc` resource is created.

ec2.tf

- `data.aws_ami.amazon_linux` dynamically looks up the latest Amazon Linux 2023 AMI so the AMI ID is never hardcoded/stale.
- `aws_security_group.app_sg` allows:
 - SSH (port 22) from `var.ssh_allowed_cidr`
 - The application port (`var.app_port`, default 3000) from anywhere
 - All outbound traffic
- `aws_instance.app_server` is the EC2 instance running Twenty CRM, using the looked-up AMI, the first subnet in the default VPC, the security group above, an encrypted gp3 root volume, and an optional key pair.

ecr.tf

- `aws_ecr_repository.app_repo` creates the ECR repository for the Twenty CRM container image, with image scanning on push enabled.
- `aws_ecr_lifecycle_policy.app_repo_policy` automatically expires untagged images after a configurable number of days, keeping the repository clean.

outputs.tf

Exposes the values you'd need after a real deploy: VPC ID, subnet IDs, EC2 instance ID and public IP, security group ID, and the ECR repository URL/ARN.

terraform.tfvars.example

A copyable template of variable values. You copy this to `terraform.tfvars` (which is gitignored) and fill in real values like your key pair name and your own IP for SSH access — real values never get committed.

.gitignore

Excludes local state files, the `.terraform/` provider cache, and `terraform.tfvars`, so secrets and local state never end up in git.

4. Prerequisites

- Terraform installed ($\geq 1.5.0$)
- Internet access to `registry.terraform.io` (needed for `terraform init` to download the AWS provider plugin)
- AWS credentials are **not required** for this task, since we stop at `terraform validate` (see Section 6)

5. Step-by-Step: Running the Configuration

1. **Unzip / clone the project** and move into the folder that actually contains the `.tf` files:

```
cd terraform-crm
ls
```

You should see `main.tf`, `variables.tf`, `vpc.tf`, `ec2.tf`, `ecr.tf`, `outputs.tf`, `terraform.tfvars.example`, `.gitignore`, `README.md`.

If `ls` only shows another folder called `terraform-crm`, the zip extracted with an extra nested layer — `cd` into it again before continuing.

2. **Copy the example variables file:**

Copy-Item terraform.tfvars.example terraform.tfvars

(On macOS/Linux: cp terraform.tfvars.example terraform.tfvars)

3. **Initialize Terraform** — downloads the AWS provider and sets up the working directory:

```
terraform init
```

Expected output ends with Terraform has been successfully initialized!

4. **Validate the configuration** — checks syntax and internal consistency:

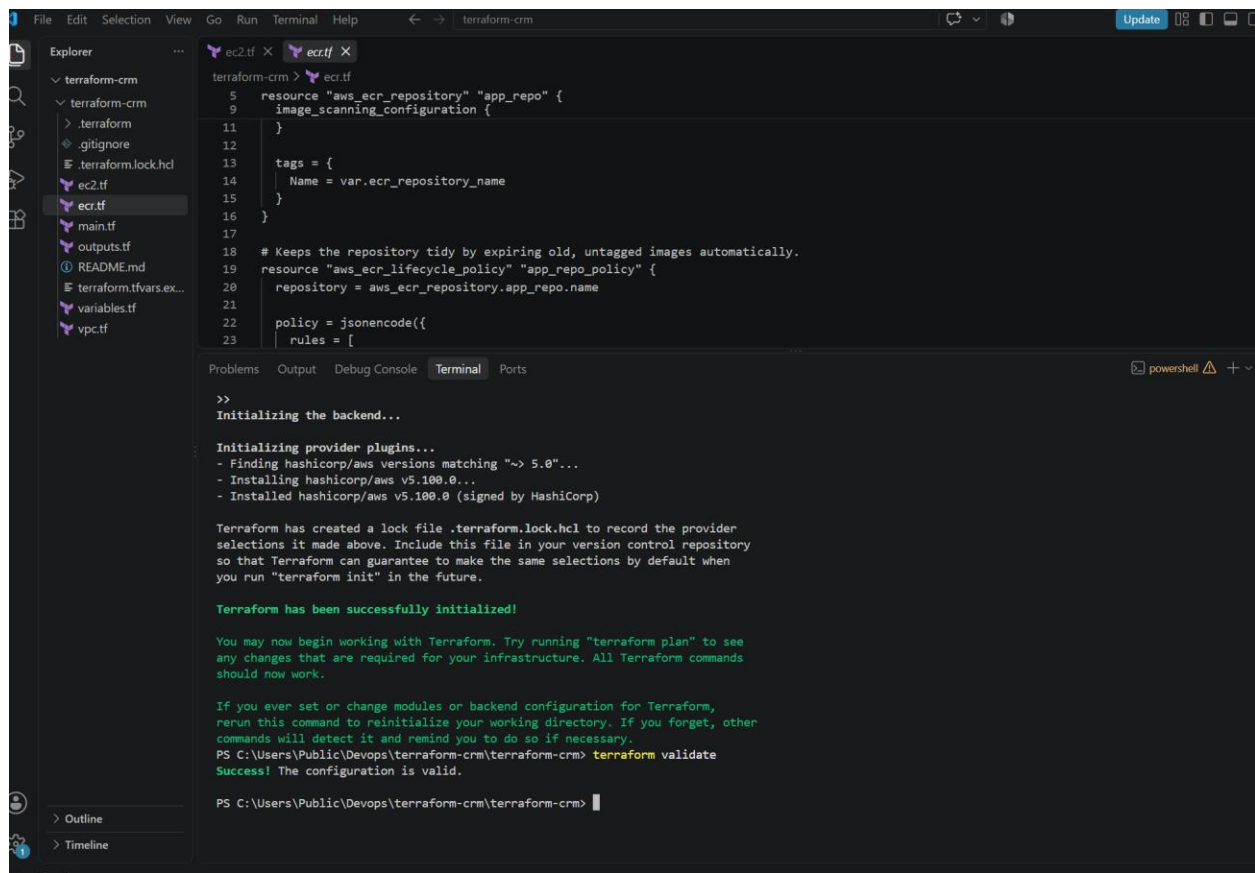
```
terraform validate
```

Expected output: Success! The configuration is valid.

That's it for this task — **no terraform plan and no terraform apply.**

6. Verification — Successful Run

Below is a screenshot confirming `terraform init` and `terraform validate` both completed successfully on the project:



- terraform init found and installed hashicorp/aws v5.100.0, matching the ~> 5.0 constraint in main.tf, and created .terraform.lock.hcl.
- terraform validate returned **Success! The configuration is valid.**

7. Git & PR Workflow

```
git checkout -b <your-name>-11
git add terraform-crm/
git commit -m "Task 11: Terraform config for VPC/EC2/ECR"
git push origin <your-name>-11
```

Then open a pull request against the devops-crm-project repository from the <your-name>-11 branch.

8. Summary

Requirement	Status
Proper Terraform project structure	✓
AWS provider configured for us-east-1	✓
VPC (existing/default), EC2, ECR resources	✓
Variables used instead of hardcoding	✓
Outputs for important resource info	✓
Clean, well-commented configuration	✓
terraform init / validate run successfully	✓ (run locally, see Section 5)